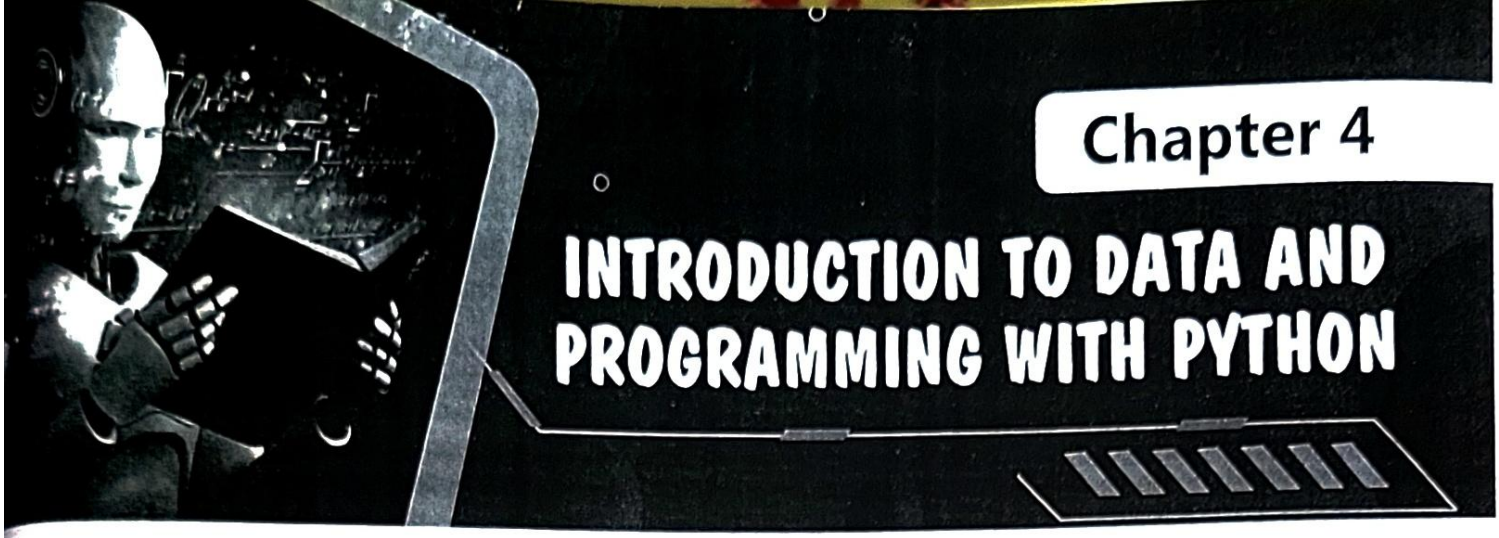


Chapter 4

INTRODUCTION TO DATA AND PROGRAMMING WITH PYTHON

Learning Outcomes

- Modules and Packages
- Tuples in Python
- Some More Programs
- Lists in Python
- Difference between List and Tuple
- Strings

Modules and Packages

A module is a python file containing a collection of Python statements, functions and global variables. It is created with an extension .py. A collection of relevant modules saved under the same directory and a name is called a Package. There are various packages related to various purposes available for free to be used in Python.

Scope and Uses of Packages

Since a package organizes related modules, sub-packages, and resources into a hierarchical structure it provides a large set of predefined functions, classes, and methods for various tasks.

The scope of the packages lies in:

Organizing code: It organises the related modules into a hierarchical structure, making it more manageable and easier to navigate.

Code reusabiltiy: Since the code is created once and can be used by anybody. Thus it provides the feature of reuseability of the code and eliminating the changes of duplicating code.

Distribution and installation: Packages allow you to distribute and install code as a unit. You can package your code and resources into a distributable package format like .tar.gz file. This makes it easier to share your code with others or deploy it on different systems.

#Coding & Computational Thinking

▶ Video Session

Scan the QR code or visit the following link to watch the video:

Modules in Python Explained | Python Built in Modules | Python Tutorial for Beginners

https://www.youtube.com/watch?v=1oFneicTaII

Modules and Libraries

Modules in Python are individual files that contain code, functions, or classes for code organisation and reusability. They can be imported into other files using the import statement. whereas, Libraries in Python are collections of precompiled code, functions, or resources that provide additional functionality to Python developers. They solve specific problems or offer specific capabilities and can be installed and imported into Python projects.

Modules help organize and reuse code within a program, while libraries extend Python's capabilities by providing ready-made solutions for various tasks or domains.

Let us now study in detail the use of some of these packages:

NumPy ('Numerical Python')

NumPy is a powerful open-source scientific package that stands for 'Numerical Python'. It uses mathematical and logical operations for handling large datasets through powerful data structure-n-dimensional arrays that also speeds up data processing. NumPy is the first step in learning to become a Python data scientist in the future. Various other libraries like Pandas, Matplotlib, and Scikit-learn are built on using some concepts of this magical library. It can also be easily interfaced with other Python packages and provides tools for integrating with other programming languages like C, C++ etc.

If you are using basic Python installed through https://www.python.org website then the NumPy package is not included by default. You need to install it separately.

NumPy can be installed by typing following command:

```
pip install NumPy
```

Once it is installed, it can be readily used in any Python code by using import keyword as shown below:

Numpy can also be imported into the Jupyter Notebook by using the given statement :

```
>>> import numpy                    # this will import the complete numpy
                                    # package
OR
>>> import numpy as npy             # this will import numpy and referred
                                    # as npy
OR
>>> from .numpy import array        #this will import ONLY arrays
                                    #from whole numpy package
OR
>>> from numpy import array as ary   #this will import ONLY
                                     #arrays and referred as ary
```

Arrays are an ordered collection of values of the same data types that can be arranged in one or more dimensions. They can be numbers, characters, Booleans, etc. The elements are referred to using index numbers(position) that start with 0 . Almost all programming languages support arrays in one form or another.

An array of one dimension is called a Vector, an array having two dimensions is called a Matrix and an array with multiple dimensions is called n-dimension array.

In NumPy we can create n-dimensional arrays and are considered as an alternative to Python lists because they allow faster access in reading and writing items effectively and efficiently. The NumPy library has a large set of built-in functions available in the form of modules and packages for creating, manipulating, and transforming NumPy arrays.

So if we compare NumPy-Arrays and Python-List then:

- Array is a collection of homogeneous values whereas List as studied in class 9 - is a collection of heterogeneous values.
- In Arrays data of one type does not support data of another type whereas in List it works perfectly by using data of one type by converting into another datatype.
- Arrays can be accessed only through package -NumPy and occupies less memory space whereas List occupies more memory space and can be accessed directly in Python without any package support.
- In arrays the mathematical operators can be directly used whereas in list the mathematical operators cannot be used directly on it instead need to be used separately on individual elements.
- Arrays are mainly used for mathematical operations where Lists are mainly used for data management.
- Syntax of creating array is:
```
import numpy
marks=numpy.array([34,23,41,42])
```
Syntax of creating a list is:
```
marks=[34,23,41,42]
```

Creating an Array using NumPy:

- Creating one dimensional array:

```
import numpy
rollno = numpy.array([1, 2, 3])
print(rollno)
```

Output:

```
[1 2 3]
```

- Create a sequential 1 D array with values as multiples of 10 from 10 to 100:

```
import numpy as np
a = np.arange(10,101,10)
print(a)
```

Output:

```
 [ 10  20  30  40  50  60  70  80  90 100]
```

Another example :

```
>>>import numpy as np
>>> student = np.array([11,'Amita',92.5])
>>> student
```

- Creating 1 D array with 4 random values:

```
import numpy as np
a = np.random.random(4)
print(a)
```

Output:

```
[0.9290503  0.89991053 0.7563874  0.89570953]
```

- Creating a 3X4 array with random integer values less than 10:

```
import numpy as np
a = np.random.randint(10, size=(3,4))
print(a)
```




Output:

```
[[3 1 4 1]
 [7 6 0 1]
 [4 2 2 9]]
```

- Creating two dimensional array of 3 rows and 4 columns with all ones as value :

```
import numpy as np
a = np.ones((3,4))
print(a)
```

Output:

```
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]
```

- Creating two dimensional array of 3 rows and 3 columns with all zeroes as value:

```
import numpy as np
a = np.zeros((3,3))
print(a)
```

Output:

```
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
```

- Creating two dimensional array of 3 rows and 4 columns with all six as value:

```
import numpy as np
a = np.full((3,4),6)
print(a)
```

Output:

```
[[6 6 6 6]
 [6 6 6 6]
 [6 6 6 6]]
```

- We can create two dimensional (2-D) arrays by passing nested lists to the array() function.

```
import numpy as np
a = np.array([11,'Arshia',93],[15,'Riddhi',95])
print(a)
```

Output:

```
[[11,'Arshia',93]
 [15,'Riddhi',95]]
```

Task

#Experiential Learning

Type "import this" in your Python IDLE and you will find a poem written by Tim Peters, a major contributer to the Python community.

Using Mathematical operators on integer array:

Operator	Syntax	Output

** (exponential)	`import numpy as np`
	`marks1 = np.array([1,2,3,4])`
	`marks2=np.array([2,1,2,1])`
	`print(marks1**marks2)`
	`print("marks1 to the power of 3 is :",marks1**3)`
		`[1 2 9 4]`
		`marks1 to the power of 3 is : [ 1 8 27 64]`

+ (Addition)	`import numpy as np`
	`marks1 = np.array([7,8,5,4])`
	`marks2=np.array([5,6,4,7])`
	`print(marks1+marks2)`
	`print("2 marks extra: ",marks1+2)`
		`[12 14 9 11]`
		`2 marks extra : [ 9 10 7 6]`

- (subtraction)	`import numpy as np`
	`marks1 = np.array([7,8,5,4])`
	`marks2=np.array([5,6,4,7])`
	`print(marks1-marks2)`
	`print("2 marks deducted :",marks1-2)`
		`[ 2 2 1 -3]`
		`2 marks deducted : [5 6 3 2]`

* (Multiplication)	`import numpy as np`
	`marks1 = np.array([7,8,5,4])`
	`marks2=np.array([5,6,4,7])`
	`print(marks1*marks2)`
	`print("2 marks multiplied :",marks1*2)`
		`[35 48 20 28]`
		`2 marks multiplied : [14 16 10 8]`

/ (divide)	`import numpy as np`
	`marks1 = np.array([10,20,30,40])`
	`marks2=np.array([2,4,3,5])`
	`print(marks1/marks2)`
	`print("2 marks divided :",marks1/2)`
		`[ 5. 5. 10. 8.]`
		`2 marks divided : [ 5. 10. 15. 20.]`

// (floor division)	`import numpy as np`
	`marks1 = np.array([10,20,30,40])`
	`marks2=np.array([2,4,3,5])`
	`print(marks1//marks2)`
	`print("2 marks divided using floor division:",marks1//2)`
		`[ 5  5 10  8]`
		`2 marks divided using floor division: [ 5 10 15 20]`

% (Remainder of division)	`import numpy as np`
	`marks1 = np.array([10,20,30,40])`
	`marks2=np.array([1,2,3,4])`
	`print(marks1%marks2)`
	`print("remainder of division:",marks1%3)`
		`[0 0 0 0]`
		`remainder of division: [1 2 0 1]`

Some of the basic mathematical operations can also be done by using built-in functions like:

1. Addition: np.add()
2. Subtraction: np.subtract()
3. Multiplication: np.multiply()
4. Division: np.divide()
5. Remainder: np.remainder()

Some important attributes and functions of NumPy Array:

Type	Function	Example	Output

Type of an object i.e. array	type(ARR)	`import numpy as np`
		`ARR = np.array([1,2,3,4])`
		`print(type(ARR))`
			`<class 'numpy.ndarray'>`

gives the dimensions of an array as an integer value. Arrays can be 1-D, 2-D or n-D.	ARR.ndim	`import numpy as np`
		`ARR = np.array([[1,2,3,4], [3,4,5,6]])`
		`print(ARR.ndim)`
			`2`

Shape of an array i.e. length of the array of each dimension	ARR.shape	`import numpy as np`
		`ARR = np.array([[1,2,3,4], [3,4,5,6]])`
		`print(ARR.shape)`
			`(2, 4)`

Size of an array i.e. counts the total number of elements	ARR.size	`import numpy as np`
		`ARR = np.array([[1,2,3,4], [3,4,5,6]])`
		`print(ARR.size)`
			`8`

Datatype of elements stored in the array	ARR.dtype	`import numpy as np`
		`ARR =`
		`np.array([[1,2,3,4],`
		`[3,4,5,6]])`
		`print(ARR.dtype)`
			`int64`

Maximum value in the element of the array	ARR.max()	`import numpy as np`
		`ARR =`
		`np.array([[10,34,45,`
		`23,12]])`
		`print(ARR.max())`
			`45`

Row wise & column wise maximum value	ARR.max(axis=1) for row
	ARR.max(axis=0) for column
		`import numpy as np`
		`ARR =`
		`np.array([[11,2,13,4],`
		`[3,4,5,6]])`
		`print("Rowwise max :",`
		`ARR.max(axis=1))`
		`print("Column wise max :",`
		`ARR.max(axis=0))`
			`Rowwise max : [13 6]`
			`Column wise max : [11`
			`4 13 6]`

Row wise & column wise minimum value	ARR.min(axis=1) for row
	ARR.min(axis=0) for column
		`import numpy as np`
		`ARR =`
		`np.array([[11,2,13,4],`
		`[3,4,5,6]])`
		`print("Rowwise min :",`
		`ARR.min(axis=1))`
		`print("Column wise min :",`
		`ARR.min(axis=0))`
			`Rowwise min : [2 3]`
			`Column wise min : [3`
			`2 5 4]`

Sum of all values in the given array	ARR.sum()	`import numpy as np`
		`ARR =`
		`np.array([[11,2,13,4],`
		`[3,4,5,6]])`
		`print("Row Wise sum :",`
		`ARR.sum(axis=1))`
		`print("Column wise sum :",`
		`ARR.sum(axis=0))`
		`print("sum is :",`
		`ARR.sum())`
			`Rowwise sum : [30 18]`
			`Column wise sum : [14`
			`6 18 10]`
			`sum is : 48`



Sorting the array	ARR.sort()	a = np.array([12, 4, -10, 23, 29, 15, -1, 45, 33, 37, -14]) # Creating a 1-D Numpy array
		print(np.sort(a)) # Printing the sorted numpy array
		# We can also sort array row wise and column wise!
		b = np.array([[-9, 5, 18, 9, 12], [10, 11, 3, -5, -10]]) # Creating a 2-D Numpy array
		print(np.sort(b, axis = 1))
		# Axis = 1 performs the sorting function row-wise
		print(np.sort(b, axis = 0))
		# Axis = 0 performs the sorting function columns-wise
			[-14 -10 -1 4 12 15 23 29 33 37 45]
			[[-9 5 9 12 18]
			[-10 -5 3 10 11]]
			[[-9 5 3 -5 -10]
			[10 11 18 9 12]]

Pandas(PANel Data)

Panda is an open-source Python library used for data manipulation and data analysis. It provides a very strong feature of using three important data structures - Series (1-dimensional),DataFrame (2-dimensional) and Panel(3--dimensional) for smooth processing and analysis of data, regardless of its origin. The data actually need not be labelled at all to be placed into a Pandas data structure.

Pandas was created by Wes McKinney in 2008 and has derived its name from both "Panel Data", and "Python Data Analysis" which means using a statistical method of analysing the data taken from the observations about different cross sections over the period of time.

Pandas libraries are built on NumPy so to work in Pandas the prerequisite is to get familiar with NumPy and install it. It gives us a single, convenient place to do most of our data analysis and visualisation work.

Data required for Pandas can be taken as :

- Tabular data with heterogeneously-typed columns, as in an SQL table or Excel spreadsheet
- Ordered and unordered (not necessarily fixed-frequency) time series data.
- Arbitrary matrix data (homogeneously typed or heterogeneous) with row and column labels.
- Any other form of observational / statistical data sets.

Installing Pandas

To install Pandas from command line, we need to type in:

```
pip install pandas
```

All libraries including NumPy and Pandas can be installed only when Python is already installed on that system.

Data Structures in Pandas

Two commonly used data structures in Pandas are:

- Series
- DataFrames

Series

Series is a one-dimensional array that can store data of any type like integer, string, float, python objects, etc. We can also say that the Pandas series is just like a column of a spreadsheet.

The values can be referred to by using data axis labels also called index. Indexes are of two types: *positional index* and *labelled index*. Positional indexes are integers starting with default 0 whereas labelled indexes are user defined labels that can be of any datatype and can be used as an index.

Pandas Series can be created by loading the datasets from existing storage like SQL Database, CSV file, and Excel file. Pandas Series can also be created from the lists, dictionary, and from any other scalar value.

Different ways of creating Series are:

- **Creating an empty series**

```
import pandas as pd
Emp=pd.Series()
```

- **Creating a Series from a NumPy array**

```
import pandas as pd
import numpy as np
data = np.array([10, 30, 50])
s1 = pd.Series(data)
print(s1)
```

Output:

```
0   10
1   30
2   50
dtype: int64
```

- **Creating series using labelled index**

```
import pandas as pd
friends = pd.Series(["Rohan", "Susan", "James"], index=[11,22,33])
print(friends)
```

Output:

```
11     Rohan
22     Susan
33     James
dtype: object
```

- **Creating a Series from a Python list**

```
import pandas as pd
cities = ['Delhi', 'Mumbai', 'Chennai', 'Kolkata']
s2 = pd.Series(cities)
print(s2)
```




Output:

```
0     Delhi
1    Mumbai
2   Chennai
3   Kolkata
dtype: object
```

- **Creating series using labelled index**

```
import pandas as pd
month=["June", "August", "October", "December"]
s4 = pd.Series(month, index = [6,8,10,12])
print(s4)
```

Output:

```
6      June
8    August
10  October
12 December
dtype: object
```

- **Creating a Series from a Python dictionary**

```
import pandas as pd
month={6:"June", 8:"August", 10:"October", 12:"December"}
s5 = pd.Series(dict)
print(s5)
```

Output:

```
6     June
8    August
10  October
12 December
dtype: object
```

- **File access using Panda**

Pandas can be accessed by using import :

```
import pandas as pd
```

To save the csv file into a variable :

```
s= pd.read_csv("Student data.csv")
```

To print the first five rows of the file:

```
print (s.head(5))
```

To print the first ten rows of the file:

```
print (s.head(10))
```



To sort the records in the file:

```
s1 = s.sort_values(by='Name',ascending = False)
print(s1.head(5))
```

- **Accessing the element of the Series**

The elements can be accessed using either positional index or labelled index.

For example:

```
import pandas as pd
Marks = pd.Series([96,92,91,93],index=["Shashi","Sonali","Neha,"Rajeev"])
print("Using Labelled index ", Marks["Sonali"])
print("Using Positional index ", Marks[1])
```

Output:

```
Using Labelled index 92
Using Positional index 92
```

To print 2 elements :

```
print(Marks[[2,3]])
```

Output:

```
Neha   91
Rajeev 93
dtype: int64
print(Marks[["Sonali","Rajeev"]])
```

Output:

```
Sonali 92
Rajeev 93
dtype: int64
```

- **Elements can be accessed using Slicing as follows:**

print(Marks[1:3]) #excludes the value at index position 3

Output:

```
Sonali 92
Neha   91
dtype: int64
print(Marks["Sonali":"Rajeev"])  #includes the value at labelled index
```

Output:

```
Sonali 92
Neha   91
Rajeev 93
dtype: int64
```

- **To display the series in the reverse order**

```
print(Marks[::-1])
```

Output:

```
Rajeev 93
```




```
Neha    91
Sonali  92
Shashi  96
dtype: int64
```

Attributes of the Series

Attributes	Description	Example

name	A name to a series is given	`import pandas as pd`
		`import numpy as np`
		`data = np.array([10,30,50])`
		`s1 = pd.Series(data)`
		`s1.name="Rollno"`
		`print(s1)`
		`0 10`
		`1 30`
		`2 50`
		`Name: Rollno, dtype: int64`

index.name	A name to an index is given	`s1.index.name="Students"`
		`Students`
		`0 10`
		`1 30`
		`2 50`
		`Name: Rollno, dtype: int64`

value	Returns the list of values in the series	`print(s1.values)` `[10 30 50]`
size	Returns the number of value in the series	`print(s1.size)` `3`
empty	Returns True if the series is empty.	`print(s1.empty)` `False`

Built in Functions in Series

Function	Description	Example

head(n)	Returns the first n members of the series. If n not specified then by default first five members are displayed.	`import pandas as pd`
		`friends =`
		`pd.Series(["Rohan","Susan","James",`
		`"Riya","Sumit","Abhinav","Vihaan"]`
		`index[11,22,33,44,55,66,77])`
		`print(friends.head(2))`
		`11 Rohan`
		`22 Susan`
		`dtype: object`

tail(n)	Returns the last n members of the series. If n is not specified then by default the last five members are displayed.	import pandas as pd friends = pd.Series(["Rohan","Susan","James", "Riya","Sumit","Abhinav","Vihaan"] index[11,22,33,44,55,66,77]) print(friends.tail()) 33 James 44 Riya 55 Sumit 66 Abhinav 77 Vihan dtype: object
count()	It counts the total non NaN values in a series	import pandas as pd friends = pd.Series(["Rohan","Susan","James", "Riya","Sumit","Abhinav","Vihaan"] index[11,22,33,44,55,66,77]) print(friends.count()) 7

DataFrames

A DataFrame is a two-dimensional labelled heterogeneous data structure that contains rows and columns. It has both a row and column index. The data arranged in the form of a row and a column resembles the data arranged in a tabular form in a table of a database.

Creating a DataFrame

- **Creating an empty DataFrame**

```
import pandas as pd
Emp = pd.DataFrame() print(Emp)
Empty DataFrame
Columns: []
Index: []
```

- **Creating a DataFrame from Numpy Array**

```
import numpy as np
import pandas as pd
a1 = np.array([10,20,30])
a2 = np.array([11,22,33])
f1=pd.DataFrame(a1)
f2=pd.DataFrame([a1,a2])
print(f1)
print(f2)
    0
0   10
```




```
1    20
2    30
     0    1    2
0    10   20   30
1    11   22   33
```

- **With column index**

```
import numpy as np
import pandas as pd
a1 = np.array([10,20,30])
a2 = np.array([11,22,33])
f2=pd.DataFrame([a1,a2],columns=['R1','R2','R3'])
print(f2)
       R1   R2  R3
0      10   20  30
1      11   22  33
```

- **Creating a DataFrame using a Python List**

```
import pandas as pd
cities = ['Delhi', 'Mumbai', 'Chennai', 'Kolkata']
f1 = pd.DataFrame(cities)
print(f1)
          0
0     Delhi
1    Mumbai
2   Chennai
3   Kolkata
```

- **Creating a DataFrame using a Python dictionary**

```
import pandas as pd
d1={'Name':['Riya','Sonali', Sneha],'Marks':[92,90,89]}
f2=pd,DataFrame(d1,index=[11,22,33])
print(f2)
       Name  Marks
11     Riya     92
22   Sonali     90
33    Sneha     89
```

Basic Operations in a DataFrame

- ***Adding a new row***

```
f2.loc[44]=['Nitin',84]
print(f2)
```

```
       Name  Marks
11     Riya     92
22   Sonali     90
33    Sneha     89
44    Nitin     84
```

- **Adding a new column**

```
f2['Section']=['A','E','C']
print(f2)
```

```
       Name  Marks    Section
11     Riya     92          A
22   Sonali     90          E
33    Sneha     89          E
44    Nitin     84          C
```

- **Modifying an existing Row**

```
f2.loc[33]=['Sneha',95,'C']
print(f2)
```

```
       Name  Marks    Section
11     Riya     92          A
22   Sonali     97          E
33    Sneha     95          C
44    Nitin     89          C
```

- **Modifying an existing column**

```
f2['Marks']=[92,97,91,89]
print(f2)
```

```
       Name  Marks    Section
11     Riya     92          A
22   Sonali     97          E
33    Sneha     91          E
44    Nitin     89          C
```

- **Deleting a Row**

To delete a row or a column, the .drop() method is used with the DataFrame. The parameter axis=0 is used for deleting a row and axis=1 for deleting a column.

```
f2=f2.drop(22,axis=0)
print(f2)
```

```
       Name  Marks    Section
11     Riya     92          A
33    Sneha     89          E
44    Nitin     84          C
```




- **Deleting a Column**

```
f2=f2.drop('Marks', axis=1)
print(f2)
```

```
       Name  Section
11     Riya        A
33    Sneha        E
44    Nitin        C
```



Task

✓#Coding & Computational Thinking

Find out what happens when you add the following statements in a DataFrame created earlier:

a. f2.loc[22]

b. f2[:] = 0

c. f2.loc[33]=0

Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

Pandas can perform some of the important functions like:

- It provides a fast and efficient way to manage and explore data using Series,DataFrames and Panels which makes it really fast and suitable for data science.
- Missing Data represented as NaN and of any data type is very easily and efficiently handled.
- Columns can be easily inserted, modified and deleted from DataFrame and Panel.
- Objects can be explicitly aligned by the users to a set of labels, or automatically handled by data structures for smooth processing and data analysis.
- Organisation and labelling of data are perfectly taken care of by the intelligent methods of slicing, alignment and indexing.
- It provides support for cleaning of data needed for processing.
- It provides strong support for reshaping, merging and joining various datasets with extreme efficiency.

SciPy(Scientific Python)

SciPy is a free, open source Python library used for scientific and technical Computation. It finds its great importance in the field of physics, engineering, mathematics, biology, and data science. SciPy is built on the Numpy extension of Python.

SciPy provides a wide range of modules and submodules that helps in the area of scientific computing. Some of the important modules in SciPy are:

- **scipy.optimize:** This module has functions for optimisation problems, including root finding, curve fitting, minimisation, and linear programming.
- **scipy.integrate:** It provides mathematical integration techniques by using functions for integrating ordinary differential equations (ODEs), solving differential algebraic equations (DAEs), and performing numerical quadrature.



- **scipy.interpolate:** This module deals with interpolation and approximation techniques, such as splines, B-splines, and radial basis functions.
- **scipy.linalg:** This module deals with linear algebra - matrix factorisation, solving linear systems of equations, eigenvalue problems, and singular value decomposition (SVD).
- **scipy.signal:** This module deals with signal processing tasks - filtering, spectral analysis, convolution, correlation, and wavelet transformations.
- **scipy.stats:** This module deals with a wide range of statistical distributions, statistical functions, hypothesis testing, and statistical modeling tools.
- **scipy.spatial:** This module deals with spatial data structures and algorithms, including distance computations, clustering, nearest neighbors, and spatial transformations.
- **scipy.fftpack:** This module deals with functions for Fast Fourier transforms (FFT) and related operations for efficient spectral analysis.

The programming in SciPy is beyond the scope of this book.

Installing SciPy

The command that installs SciPy is:

```
pip install scipy
```

Assuming that Python and pip are already installed in the computer.

Statistical Learning with Python

Now we all have understood that Data Sciences deals with data analysis and data manipulation . But this numeric & alpha-numeric data analysis and manipulation is not possible without the intervention of Mathematical Statistics.

Python with supported libraries like NumPy, Matplotlib etc have a lot of pre-defined functions that implement Mathematical statistics without getting into the hassle of doing the calculations and creating the formulas or equations to find out the results. All we need to do is write that function and pass on the data to it. It's that simple!

Let us take a look at some basic statistical tools used in Python:

- **Mean**

  It is the average of numeric data in a given dataset. So we add all of the numbers together and divide by the number of data in the given set.

  It is calculated as :

  Mean = sum of all values / no. of values

  To calculate mean in Python :

```
import statistics
marks=[45,34,41,46,47,39,38,48,45,34,41,39,39]
m=statistics.mean(marks)
print("the average marks of the class :",round(m,2))
```

  Output:

```
the average marks of the class : 41.23
```

- **Median**

  When the data is arranged in an ascending order then It is the middle number in a given dataset. If there are two middle numbers, taking the mean of these two numbers will give the median.

  For odd number of dataset:



$$\frac{(N+1)^{th}}{2}$$

For even number of dataset:

$$\frac{\frac{N^{th}}{2}\text{term}+\left(\frac{N}{2}+1\right)\text{term}}{2}$$

To calculate median in Python :

```
import statistics
marks=[45,34,41,46,47,39,38,48,45,34,41,39,39]
m=statistics.median(marks)
print("the middle value of the sorted marks in class :",m)
```

Output:

```
the middle value of the sorted marks in class : 41
```

- **Mode**

It is the most frequent value in a given dataset.

There is no formula for this:

Mode = The value in the data set that occurs most frequently

To calculate mode in Python :

```
import statistics
marks=[45,34,41,46,47,39,38,48,45,34,41,39,39]
m=statistics.mode(marks)
print("the most frequent marks :",m)
```

Output:

```
the most frequent marks : 39
```

- **Standard Deviation**

It is the measure of dispersion of a set of data from its mean. The higher the dispersion of the data, the greater is the standard deviation and vica versa.

The formula for calculation:

$$\sigma=\sqrt{\frac{\sum(x-\text{mean})^2}{n}}$$

To calculate Standard Deviation in Python:

```
import statistics
marks=[45,34,41,46,47,39,38,48,45,34,41,39,39]
m=statistics.stdev(marks)
print("the standard deviation of marks :",round(m,2))
```

Output:

```
the standard deviation of marks : 4.66
```

- **Variance**

It means how far each number in the given dataset is from the mean value. It is calculated as the average of the squares of the differences between an individual value and the expected value or average squared deviation of each number from the mean of a given data set.

The formula for calculation:

$$\text{variance} = \sigma^2 = \frac{\sum(x_i - \mu)^2}{n}$$

To calculate Variance in Python:

```
import statistics
marks=[45,34,41,46,47,39,38,48,45,34,41,39,39]
m=statistics.variance(marks)
print("the variance of marks :",round(m,2))
```

Output:

```
the variance of marks : 21.69
```

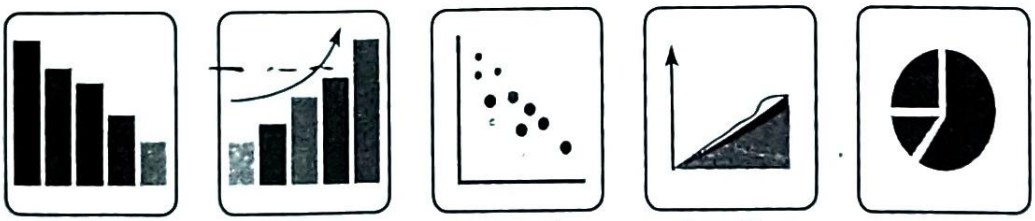
Matplotlib

Matplotlib was created by John D. Hunter in 2003. It is a free and open source, data visualisation library used for plotting graphs and visualisation in Python built on NumPy arrays and can be downloaded from https://www.matplotlib.org. Using Matplotlib, with just a few lines of code we can generate graphs of any kind like plots, histograms, bar charts, scatterplots, etc.

It comes with a wide variety of plots but the most used module of Matplotib is Pyplot which is used for creating 2D plots of arrays.

Visuals have more impact on human brains. So these plots created using Matplotlib help us understand the trends, patterns, correlations of the data. They give us the quantitative information of data in a visual form. After the plots are created, you can change the style, look and make them more descriptive and communicable.

Some types of graphs that we can make with this package are listed below:



Bar Graph | Histogram | Scatter Plot | Area Plot | Pie Plot

Installing Matplotlib

- To install it using pip:

```
python -m pip install -U pip
python -m pip install -U matplotlib
```

Or

```
pip install matplotlib
```

- To install using Anaconda

```
conda install matplotlib
```

Or

```
conda install -c conda-forge matplotlib
```

For plotting using Matplotlib, we need to import its Pyplot module as follows:

```
import matplotlib.pyplot as plt
```